

Les embouteillages sont parfois des freins à la mobilité pour les personnes utilisant leurs voitures. La gestion des carrefours et des feux tricolores est en général faite par rotation où chaque feu passe au vert à son tour durant un temps déterminé. On se propose ici d'étudier un système "intelligent" dont le fonctionnement serait le suivant:

- Un **carrefour** est composé de plusieurs **routes**. Sur chacune de ces routes, on place un **feu**.
- Un **carrefour** peut posséder plusieurs **routes**.
- Chaque route a un **nom**, un **feu**, ainsi qu'un **nombre de véhicules** attendant au feu.
- Chaque **feu** possède une **couleur** (Rouge, Vert et Orange).

Il est possible de créer un diagramme pour chacune des classes présentées:

Nom de la classe: Route	Nom de la classe: Feu	Nom de la classe: Carrefour
*****	*****	*****
Attributs:	Attributs:	Attributs:
- nom (str)	- couleur (str)	- route (list)
- nb_vehicules (int)	- couleur_suivante (dict)	une liste de routes
- feu (de type Feu)	*****	*****
*****	Méthodes:	Méthodes:
Méthodes:	- passer_suivant()	- route_max()
- ajouter_vehicule(n)		- adapter_circulation()
		- passage_voitures()
		- passages_voitures_rec()

Il est donc possible de créer un **carrefour** par exemple à partir du code suivant:

```

</> Code Python
1 # Création des routes
2 R1 = Route('R1', 'Rouge', 4) # Route R1 avec un feu Rouge et 4 véhicules
3 R2 = Route('R2', 'Vert', 2)  # Route R2 avec un feu Vert et 2 véhicules
4 R3 = Route('R3', 'Rouge', 6) # Route R3 avec un feu Rouge et 6 véhicules
5
6 # Création du carrefour
7 C = Carrefour([R1, R2, R3])

```

1. Un feu tricolore a un cycle de couleur donné par: Rouge -> Vert -> Orange -> Rouge.

Ecrire la méthode **passer_suivant** qui permet de simuler ce phénomène cyclique et changer la couleur du feu pour la couleur suivante. Nous utiliserons obligatoirement un dictionnaire dont les clés sont les couleurs actuelles et les valeurs les couleurs suivantes.

Par exemple, on pourra avoir le comportement suivant:

```

</> Code Python
1 R1 = Route('R1', 'Rouge', 4) # Route R1 avec un feu Rouge et 4 véhicules
2
3 R1.feux.passer_suivant() # Le feu de R1 passe de Rouge à Vert
4 R1.feux.passer_suivant() # Le feu de R1 passe de Vert à Orange
5 R1.feux.passer_suivant() # Le feu de R1 passe de Orange à Rouge

```

On pourra tester cette méthode en décommentant la ligne **#test_classe_feu()**.

Appel	Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.
-------	---

2. Un **carrefour** est un ensemble de routes reliées entre elles. Ces carrefours sont modélisées par une liste de routes. Afin de savoir quelle route prioriser, écrire une méthode **route_max** qui renvoie la route dont le nombre de véhicules est maximal. En cas d'égalité, c'est la route créée en premier qui est prioritaire.

Par exemple, on pourra avoir le comportement suivant:

Code Python

```
1 # Création des routes
2 R1 = Route('R1', 'Rouge', 4)    # Route R1 avec un feu Rouge et 4 véhicules
3 R2 = Route('R2', 'Vert', 2)     # Route R2 avec un feu Vert et 2 véhicules
4 R3 = Route('R3', 'Rouge', 6)    # Route R3 avec un feu Rouge et 6 véhicules
5
6 # Création du carrefour
7 C = Carrefour([R1, R2, R3])
8
9 route_prioritaire = C.route_max() # Route R3 est la route prioritaire
```

On pourra tester cette méthode en décommentant la ligne `test_methode_route_max()`.

Appel2

Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.

3. Afin de rendre le passage des voitures plus efficace, on propose d'utiliser le fonctionnement suivant:

- la route prioritaire est la route sur laquelle il y a le plus de voitures présentes.
- on fait passer le feu de la route prioritaire à Vert
- on fait passer une voiture sur la route prioritaire
- on fait passer les autres feux à la couleur Rouge.

La méthode suivante permet de gérer ce phénomène, cependant, le résultat obtenu n'est pas celui désiré. Identifier l'erreur et la corriger de manière à obtenir un fonctionnement correct.

Code Python

```
1 def adapter_circulation(self):
2     ''' permet d'adapter la circulation:
3     - détermination de la route prioritaire
4     - passage au vert de la route prioritaire
5     - passage d'un véhicule sur cette route
6     - passage au rouge des autres routes
7     '''
8     route_prioritaire = self.route_max()
9     for r in self.routes:
10        if route_prioritaire == r.nom:
11            r.feux.couleur = "Vert"
12            r.ajouter_vehicules(-1)
13        else:
14            r.feux.couleur = 'Rouge'
```

Par exemple, on pourra avoir le comportement suivant:

Code Python

```
1 # Création des routes
2 R1 = Route('R1', 'Rouge', 4)    # Route R1 avec un feu Rouge et 4 véhicules
3 R2 = Route('R2', 'Vert', 2)     # Route R2 avec un feu Vert et 2 véhicules
4 R3 = Route('R3', 'Rouge', 6)    # Route R3 avec un feu Rouge et 6 véhicules
5
6 # Création du carrefour
7 C = Carrefour([R1, R2, R3])
8
9 C.adapter_circulation()
10 # La route R3 sera prioritaire
11 # R1 aura un feu Rouge et 4 véhicules
12 # R2 aura un feu Rouge et 2 véhicules
13 # R3 aura un feu Vert et 5 véhicules
```

Vous pourrez tester cette méthode en décommentant la ligne `test_methode_adapter_circulation()`.

Appel3

Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.

4. Afin de gérer le passage des différentes voitures, nous allons compléter la méthode `passage_voitures_rec`.

Compléter la méthode `passage_voitures_rec` qui doit simuler le passage des voitures jusqu'à ce qu'il n'en reste plus aucune sur aucune route.

Par exemple, on pourra avoir le comportement suivant:

```
</> Code Python
1 # Création des routes
2 R1 = Route('R1', 'Rouge', 1)      # Route R1 avec un feu Rouge et 1 véhicules
3 R2 = Route('R2', 'Vert', 2)       # Route R2 avec un feu Vert et 2 véhicules
4 R3 = Route('R3', 'Rouge', 3)      # Route R3 avec un feu Rouge et 3 véhicules
5
6 # Création du carrefour
7 C = Carrefour([R1, R2, R3])
8
9 C.passage_voitures()
10 # R3 prioritaire -> R1: 1 véhicule, R2: 2 véhicules, R3: 2 véhicules
11 # R2 prioritaire -> R1: 1 véhicule, R2: 1 véhicule, R3: 2 véhicules
12 # R3 prioritaire -> R1: 1 véhicule, R2: 1 véhicule, R3: 1 véhicule
13 # R1 prioritaire -> R1: 0 véhicule, R2: 1 véhicule, R3: 1 véhicule
14 # R2 prioritaire -> R1: 0 véhicule, R2: 0 véhicule, R3: 1 véhicule
15 # R3 prioritaire -> R1: 0 véhicule, R2: 0 véhicule, R3: 0 véhicule
```

Vous pourrez tester cette méthode en décommentant la ligne `test_methode_passage_voiture()`.

Appel4	Appeler le professeur pour lui présenter votre fonction et son fonctionnement ou en cas de difficultés.
--------	---